Mini lecture · 20 minutes · after Vaswani et al. (2017)

# Attention Is All You Need, read as a design document

The 2017 paper names a constraint, makes a trade, and backs it with evidence. Every model your harness calls descends from it.

CONSTRAINT | MECHANISM | EVIDENCE | LEGACY

Explainers: ahuimanu.github.io/CIDM6333 · Figures 01, 01b, 01c

# Twenty minutes, five moves

0:00–0:01

*The run of show; the clock at left is minutes into the mini lecture*

| | | | |
|---|---|---|---|
| 1 | 0:01 | **The constraint** | Recurrence is a queue: one step per token, in order. |
| 2 | 0:04 | **The mechanism** | Scaled dot-product attention, eight heads, and position written in waves. |
| 3 | 0:10 | **The evidence** | Quality, training cost, and one change at a time (Tables 2 and 3). |
| 4 | 0:15 | **What chat models kept** | The decoder column, the mask, and the loop the figure does not draw. |
| 5 | 0:18 | **Why the harness engineer cares** | Five lines from the paper to Evolutions 1, 3, 6, 10, and 11. |

# The constraint: recurrence is a queue



*Table 1 of the paper, redrawn; n is sequence length, d is the model width*

**What the authors faced**
**Recurrence reads one token per step. Each step waits for the last; a sentence cannot spread across a GPU.**
**The trade: all positions attend to each other at once. Sequential steps O(n) → O(1); compute per layer grows with $n^2$.**

| Layer type | Complexity / layer | Sequential ops | Max path |
|---|---|---|---|
| **Self-attention** | $O(n^2 \cdot d)$ | O(1) | O(1) |
| Recurrent | $O(n \cdot d^2)$ | O(n) | O(n) |
| Convolutional | $O(k \cdot n \cdot d^2)$ | O(1) | $O(\log_k n)$ |
| Self-attention (restricted) | $O(r \cdot n \cdot d)$ | O(1) | O(n/r) |

Self-attention is cheaper than recurrence *"when the sequence length n is smaller than the representation dimensionality d, which is most often the case"* (§4). In 2017, n was a sentence of about 30 tokens and d was 512.

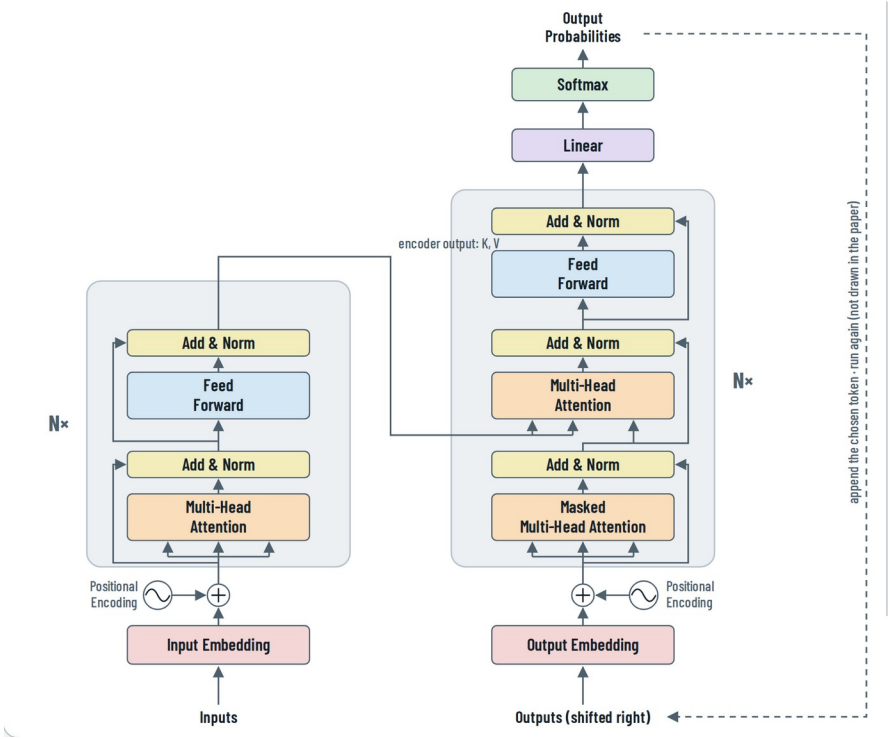
**O(n) → O(1)**
sequential steps per layer, recurrent vs. self-attention

*In 2026, n is your context window, far past d. The $n^2$ term is the cost you budget in E6.*

# Figure 1, redrawn: two stacks and a loop



*Layout, names, and color roles follow the paper; the dashed return arrow is ours (§3 says the model is auto-regressive)*




## Encoder (left)

Reads the source sentence. Every position attends to every other, in both directions.

## Decoder (right)

Writes the translation one token per pass. Its first attention layer is masked: it can look back, never ahead.

## The base model

N = 6 layers per stack, d_model = 512, 8 heads, about 65M parameters, 12 hours on one machine with 8 P100 GPUs.

## The loop

"At each step the model is auto-regressive, consuming the previously generated symbols as additional input when generating the next" (§3).

# Scaled dot-product attention: one line, three choices

0:06–0:08

*§3.2.1 and §3.2.3; the grid is computed from random vectors and shows arithmetic only*

$$\text{Attention}(Q, K, V) = \text{softmax}( QK^T / \sqrt{d_k} ) V$$

### Q, K, V come from one place

In self-attention "all of the keys, values and queries come from the same place." Each is a projection of the same input.

### Divide by √d_k

Large dot products push softmax into "regions where it has extremely small gradients." Scaling keeps the weights trainable.

### Mask the future

The decoder sets illegal connections to −∞ before softmax, so each position predicts only from earlier positions.

| | the | model | is | an | untrusted | caller |
|---|---|---|---|---|---|---|
| the | 1.00 | -∞ | -∞ | -∞ | -∞ | -∞ |
| model | 0.76 | 0.24 | -∞ | -∞ | -∞ | -∞ |
| is | 0.13 | 0.81 | 0.06 | -∞ | -∞ | -∞ |
| an | 0.16 | 0.22 | 0.09 | 0.53 | -∞ | -∞ |
| untrusted | 0.10 | 0.49 | 0.09 | 0.22 | 0.09 | -∞ |
| caller | 0.03 | 0.18 | 0.05 | 0.18 | 0.50 | 0.05 |

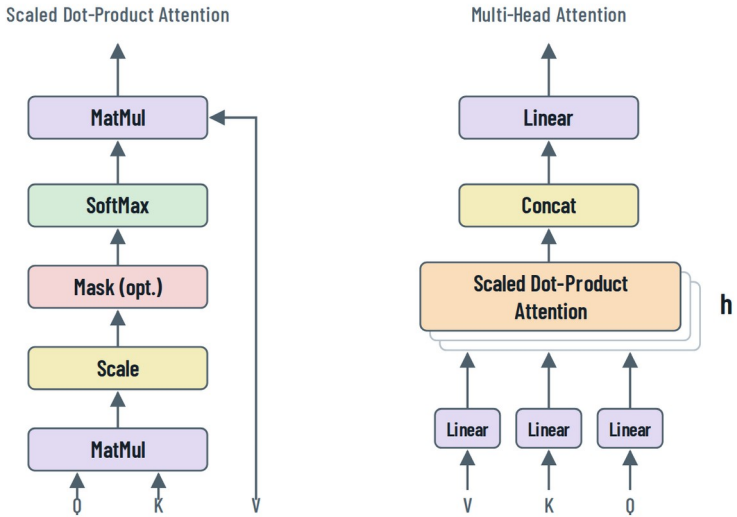
*Rows are queries, columns are keys, each row sums to 1. The striped cells are the −∞ entries. Figure 01b lets you turn scaling off and raise d_k until each row collapses onto one key.*

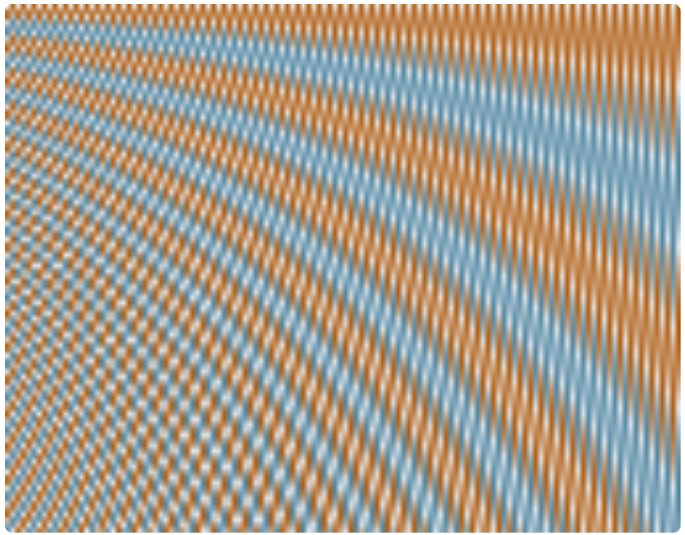
*The mask is the one detail every chat model still uses unchanged.*

# Eight heads, and word order written in waves

0:08–0:10

*Figure 2 (right) redrawn, and the sinusoidal positional encoding of §3.5*



**Multi-head:** h = 8 heads, each of width d_k = d_v = 512 / 8 = 64. Total computation matches one full-width head. Table 3 row A: one head scores 0.9 BLEU worse than the best setting, and 32 heads also drop.

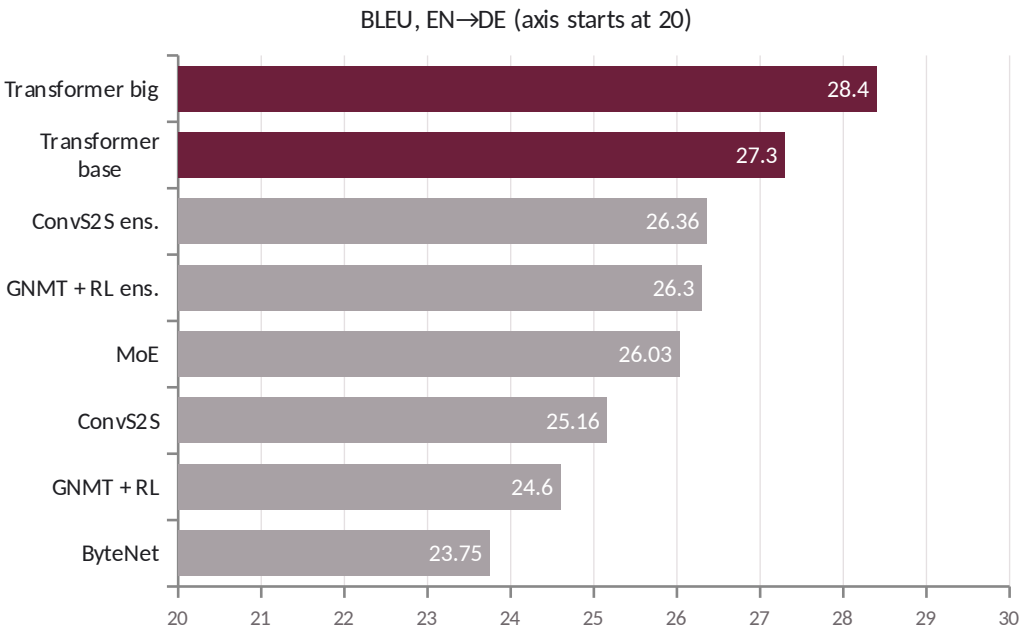
**Position:** attention alone ignores word order, so each position adds sine and cosine waves of different wavelengths. Table 3 row E: learned positions gave nearly identical results.

*Heatmap rows: positions 0–99; columns: dimensions 0–127 of 512; blue −1, amber +1.*

# Evidence 1: better translations at lower training cost

0:10–0:13

*Table 2, English-to-German, newstest2014; training cost is the paper's own estimate*




**28.4**

BLEU EN→DE (big), more than 2.0 above the best earlier result, ensembles included

**$3.3 \cdot 10^{18}$**

training FLOPs for the base model: 12 h × 8 GPUs × 9.5 TFLOPS

**41.8 / 41.0**

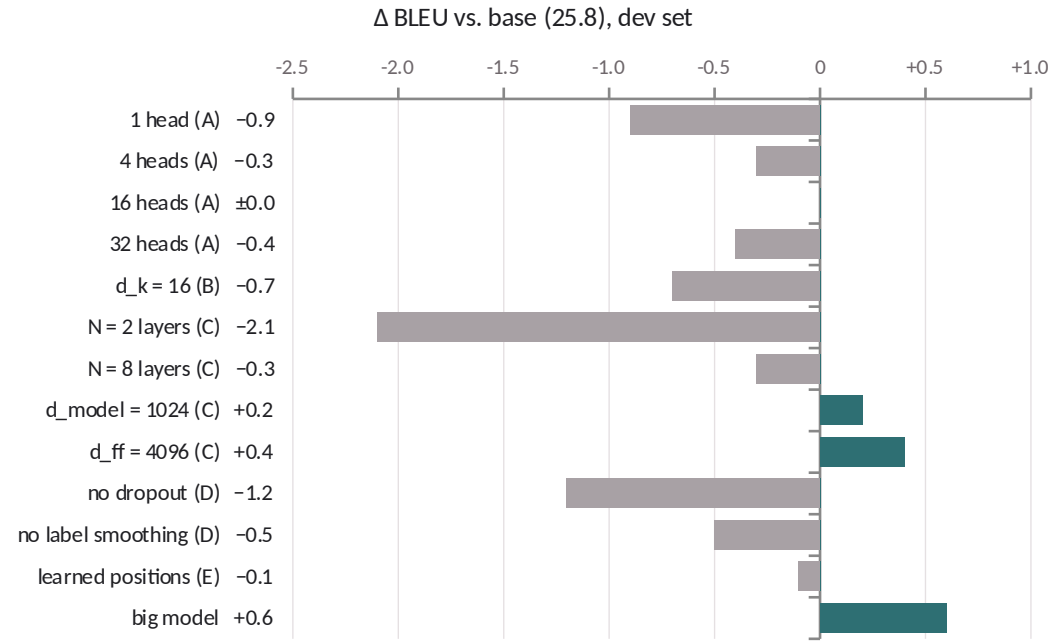
BLEU EN→FR (big): Table 2 says 41.8, the §6.1 text says 41.0

*The FLOPs figure is an estimate with its method printed beside it. That is what a receipt looks like (E11).*

# Evidence 2: one change against a fixed base

0:13–0:15

*Table 3, English-to-German dev set (newstest2013); change in BLEU from the base model's 25.8*




### What the authors concluded (§6.2)

"bigger models are better, and dropout is very helpful in avoiding over-fitting."

"reducing the attention key size d_k hurts model quality."

### Read it as an eval gate (E10)

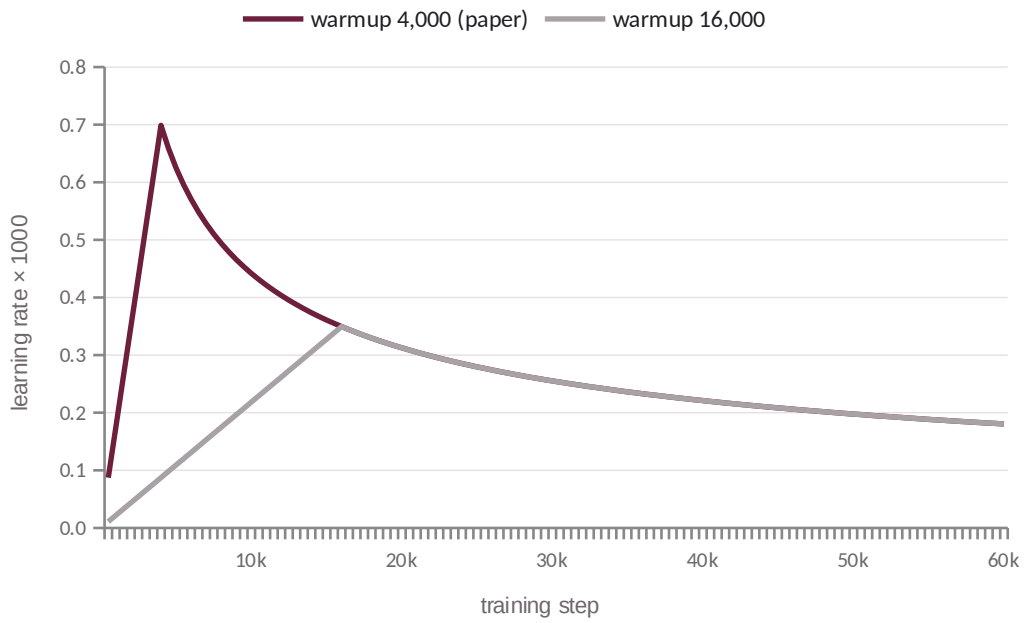
One variable per row. A fixed base. A dev set kept apart from the test set. Every row is a delta against the same baseline, which is the shape of your Sortie 10 gate.

*Before you trust a row, ask what threshold you would have pre-registered. A 0.1 change is inside run-to-run noise.*

# The recipe the results depend on

0:15–0:16

*Equation 3 (§5.3): warm up linearly, then decay with the inverse square root of the step*




$$\text{lrate} = \text{d_model}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5})$$

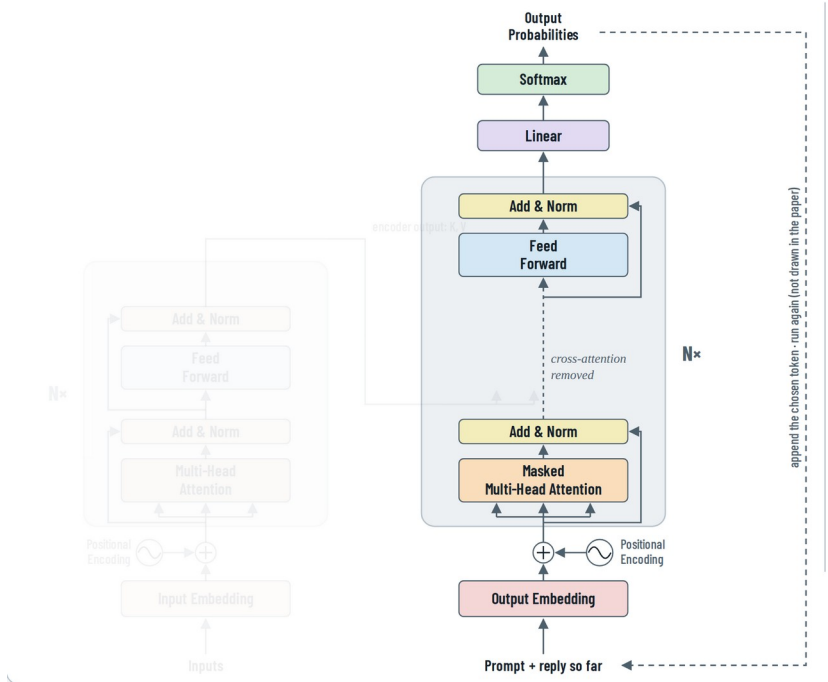
**The rest of the recipe**
**Adam, $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\varepsilon = 10^{-9}$.**
**Dropout 0.1 on sub-layers and embeddings.**
**Label smoothing 0.1 "hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score."**
**100,000 steps base, 300,000 big.**

*A result is the architecture plus its recipe. Change the recipe and the claim no longer holds as stated.*

# What chat models kept: the right-hand column

0:16–0:18

*GPT (Radford et al., 2018) and its descendants drop the encoder and the encoder–decoder attention*




### Kept

- Token embeddings and position
- Masked self-attention
- Feed-forward and Add & Norm, stacked N times
- Linear and softmax over the vocabulary

### Dropped or changed

- The encoder stack
- Encoder–decoder attention
- Beam search, in favor of sampling
- Post-norm: most later models normalize before each sub-layer

**One sequence.** The prompt and the reply become one stream of tokens. The model reads all of it and writes the next token, then the harness or the model's own loop appends that token and runs again.

*Sampling replaced beam search, which is why the same prompt can give different answers (E10).*

# From the paper to your harness

0:18–0:20

*Five lines from 2017 to this course; Figure 01, Two Loops, draws the model inside your agent loop*

| In the paper | In this course | What it means for your sortie |
|---|---|---|
| Self-attention costs $O(n^2 \cdot d)$ | E6 · context budget | Every token you add to the window raises the cost of every later call. |
| Output probabilities end Figure 1 | E1 · E3 · the harness | A tool call is text the model wrote. Your harness runs it, after the gate. |
| Sampling replaced beam search | E10 · evals as gates | One run is an anecdote. Run the golden set several times; gate on the delta. |
| Table 3: one change, fixed base, dev set | E10 · delta gate | Pre-register the threshold, keep the test set apart, change one variable. |
| FLOPs = time × GPUs × TFLOPS | E11 · receipts | Print the method beside the number so your Chase can recompute it. |

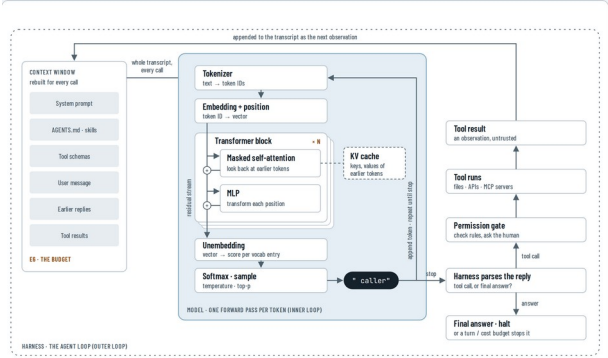



*Figure 01: the inner loop (the model, one token per pass) sits inside the outer loop (your harness).*

*The paper ends at output probabilities. Everything you build in CIDM 6333 starts at that line.*

# A design document names constraint, trade, and evidence.

Figure 01 · Two Loops: the model inside the agent loop
Figure 01b · The 2017 Transformer: Figure 1 step by step, decoder-only switch
Figure 01c · The Transformer's Evidence: Tables 2–4, the schedule, the attention pictures
ahuimanu.github.io/CIDM6333 · Vaswani et al. (2017), arXiv:1706.03762